Linear regression

Phuong Luu Vo

September 15, 2026

1 Example

Assume that the number of dengue cases and the rainfall in the last 3 years are shown in Figure 1.

$$
\begin{aligned}
X^o = &[55, 102, 169, 177, 185, 236, 252, 269, 359, 380, 386, 115,\\
&11, 16, 137, 149, 216, 234, 251, 253, 356, 364, 372, 362,\\
&36, 58, 63, 77, 282, 287, 304, 305, 370, 374, 399, 279]\\
y = &[163, 239, 227, 364, 265, 354, 325, 364, 577, 529, 544, 214,\\
&109, 9, 155, 179, 339, 434, 410, 373, 546, 486, 536, 471,\\
&189, 275, 51, 184, 293, 501, 431, 462, 457, 514, 486, 461]
\end{aligned}
$$

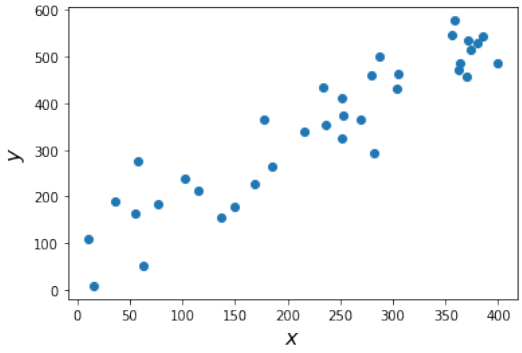


Figure 1: Dengue cases and the rainfall.

In case X^o has more than one feature, X^o is a matrix $X^o = \begin{bmatrix} (x_1^o)^T \\ ... \\ (x_N^o)^T \end{bmatrix}$, in which x_i^o is the i observation and N is the number of observations.

We will find a linear function that fits the data.

2 Linear regression as convex optimization problem

Assuming the linear regression model is given by

$$w_0 + x_i^o w_1 = x_i^T w,$$

where $x_i = [1, x_i^o]^T$.

Error (residual) between the predicted value and the true value is:

$$L_i(w) = (y_i - x_i^T w)^2.$$

Total error is given by:

$$\begin{aligned} L(w) &= \sum_{i=1}^{N} L_i = \sum_{i=1}^{N} (y_i - x_i^T w)^2 \\ &= \|y - Xw\|_2^2. \end{aligned}$$

If using ℓ_1 norm, the total error is given by:

$$\begin{aligned} L(w) &= \sum_{i=1}^{N} |y_i - x_i^T w| \\ &= \|y - Xw\|_1. \end{aligned}$$

In general,

$$L(w) = ||y - Xw||,$$

where $X = \begin{bmatrix} x_1^T \\ ... \\ x_N^T \end{bmatrix}$

We find w to minimize $L(w)$

$$\text{min. } L(w) = ||y - Xw||. \tag{1}$$

This is a convex problem.

If using ℓ_2-norm, it becomes a *least square* problem:

$$\text{min. } ||y - Xw||_2^2 = (y - Xw)^T (y - Xw). \tag{2}$$

First order condition is as follow:

$$\nabla_w (y - Xw)^T (y - Xw) = 2(X^T Xw - X^T y) = 0$$

Hence, the closed-form solution is given by

$$w^* = (X^T X)^{-1} X^T y. \tag{3}$$

3 Using CVXPY to solve the linear regression problem

Import:

```
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt
```

Import dataset:

```
X = np.array((55,102,169,177,185,236,252,269,359,380,386,115, 11,16,137,149,216,
              234,251,253,356,364,372,362, 36,58,63,77,282,287,304,305,370,374,

Y = np.array((163,239,227,364,265,354,325,364,577,529,544,214, 109,9,155,179,339
              434,410,373,546,486,536,471,189,275,51,184,293,501,431,462,457,51
```

Add 1 column to X

```
X0 = np.ones((np.shape(X)[0]))
X_new = np.column_stack(([X0,X]))
```

Objective function:

```
def obj_fn(X, Y, beta):
    return cp.pnorm(X @ weight - Y, p=2)**2
```

Define the problem and solve:

```
weight = cp.Variable(np.shape(Xnew)[1])
problem = cp.Problem(cp.Minimize(obj_fn(X_new, Y, weight)))
problem.solve()
print(weight.value)
```

Solution w^* is

```
array([79.96960019,  1.16955022])
```

Figure 2 shows the fitted line.

```
Y_est=X.dot(weight.value[1]) + weight.value[0]
plt.scatter(X, Y)
plt.plot(X, Y_est, 'r')
plt.xlabel("x", fontsize=16)
plt.ylabel("y", fontsize=16)
```

4 Ridge Regression and Lasso Regression

Understand the following contents:

- Least-quare `https://www.cvxpy.org/examples/basic/least_squares.html`

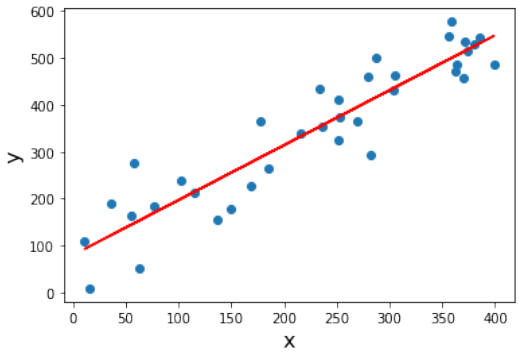


Figure 2: The fitted line.

- Ridge Regression, `https://www.cvxpy.org/examples/machine_learning/ridge_regression.html`
- Lasso regression `https://www.cvxpy.org/examples/machine_learning/lasso_regression.html`

References

[1] Stephen P. Boyd and Lieven Vandenberghe. Convex optimization. Cambridge university press, 2004.